CLASS NOTES · HANDOUT

Prompting Techniques, Fast and Slow

Derivations, code and prompt templates for the 60-minute class, organised around Kahneman's System 1 and System 2.

Parv Agarwal · AI-NLP-ML Lab, IIT Patna · Companion files: `index.html` (slides), `lab/prompting_lab.py` (code). Everything runs offline.

0. Run sheet for the class
1. Set-up: a prompt as a condition
2. Decoding: temperature and top-p
3. Zero-shot: why specification helps
4. Few-shot as Bayesian inference
5. Anchoring and contextual calibration
6. Framing and averaging correlated errors
7. Chain-of-thought and serial computation
8. Self-consistency: the full analysis
9. Decomposition
10. Tree of Thoughts and search
11. Program-aided prompting
12. Self-critique and verifiers
13. Reasoning models
14. Bias as bugs: base rates, conjunction, availability
15. Automatic prompt optimisation
16. Cheat sheet
17. Exercises with solutions
18. Running the lab
19. References

> **Kahneman in one paragraph.** *Thinking, Fast and Slow* (2011) describes two modes of thought. **System 1** is fast, automatic and associative: it produces an answer immediately, with no sense of effort. **System 2** is slow, deliberate and effortful: it does arithmetic, follows rules, and checks System 1's suggestions, but it is lazy and only engages when prompted. Most of the famous errors in the book (bat and ball, Linda, the taxi cab, anchoring, framing) are System 1 answers that System 2 failed to check. All examples below are paraphrased from the original research papers; read the book for Kahneman's own telling.

0. Run sheet for the class (60 minutes)

Time	Slides	Content	Activity
0–5	1–3	Opening, map, bat and ball	Five-second answer, show of hands
5–12	4–6	Two systems, the one equation, decoding	Temperature slider: watch $0.05 appear
12–27	7–13	Zero-shot, few-shot, number game, anchoring, calibration, framing, roles	**Live anchoring experiment** (split by roll number)
27–48	14–23	CoT, serial compute, playground, self-consistency, independence, decomposition, ToT, PAL, critique, reasoning models	17 × 24; playground predictions; Game of 24 on paper
48–54	24–27	Taxi cab, Linda, availability/RAG, auto-prompting	Guess first, then reveal; hand vote on Linda
54–60	28–31	Cheat sheet, quiz, homework, questions	Quiz: class shouts answers

Before class: open `index.html` in Chrome or Edge and press F for full screen. Press N to see speaker notes with timings. If running late, skip slides 13, 21 and 27 (one minute each).

1. Set-up: a prompt as a condition

A language model with parameters θ assigns a probability to every token sequence. By the chain rule of probability, for an output $y = (y_1, \ldots, y_n)$ given an input x:

$$p_\theta(y \mid x) = \prod_{t=1}^{n} p_\theta(y_t \mid x, y_{<t}).$$

Pretraining minimises the negative log-likelihood of text, $\mathcal{L}(\theta) = -\sum_t \log p_\theta(y_t \mid y_{<t})$. After training, θ is frozen. **The only thing a prompt engineer controls is** x (and the decoding procedure). Prompt engineering is therefore the search problem

$$x^* = \arg\max_{x \in \mathcal{V}^*} \mathbb{E}_{(q,a)\sim\mathcal{D}}\Big[m\big(\text{decode } p_\theta(\cdot \mid x \oplus q),\, a\big)\Big],$$

where $\mathcal{V}^*$ is the set of all token strings, $\oplus$ is concatenation, q a task input with reference answer a, and m a metric such as exact match. The search space is discrete and astronomically large, which is why we use heuristics (the techniques below) or automatic optimisers (§15).

Compute per token: why System 1 and System 2 map onto models

A decoder-only transformer with P parameters spends about $2P$ floating-point operations per generated token (one multiply and one add per weight), plus attention costs that grow with context length. Every token passes through the same L layers. Two consequences:

- **Fixed effort per token.** The model cannot spend more computation on a "hard" token than an easy one. This is System 1: constant, automatic effort.
- **Tokens are the only way to buy more computation.** If the model writes T intermediate tokens before answering, the answer is computed after roughly $T \cdot L$ layers of serial processing, with all intermediate results stored in the context. This is System 2: effortful, serial, using working memory.

$$\text{FLOPs}(\text{answer only}) \approx 2P\,|y|, \qquad \text{FLOPs}(\text{reason, then answer}) \approx 2P\,(|r| + |y|).$$

---

2. Decoding: temperature and top-p

The network outputs logits $z \in \mathbb{R}^{|\mathcal{V}|}$. Temperature-scaled softmax:

$$p_i(T) = \frac{e^{z_i/T}}{Z(T)}, \qquad Z(T) = \sum_j e^{z_j/T}.$$

Limits

Let $z_{(1)}$ be the largest logit, unique. Dividing numerator and denominator by $e^{z_{(1)}/T}$: $p_i(T) = e^{(z_i - z_{(1)})/T} / \sum_j e^{(z_j - z_{(1)})/T}$. As $T \to 0^+$, every exponent with $z_j < z_{(1)}$ goes to $-\infty$, so $p \to$ one-hot on the argmax (**greedy decoding**). As $T \to \infty$, every exponent goes to 0, so $p_i \to 1/|\mathcal{V}|$ (uniform).

Entropy increases with temperature (derivation)

Write $\beta = 1/T$. Then $\log p_i = \beta z_i - \log Z(\beta)$, and the entropy in nats is

$$H(\beta) = -\sum_i p_i \log p_i = \log Z(\beta) - \beta\, \mathbb{E}_p[z].$$

Two standard facts about the log-partition function: $\frac{d}{d\beta} \log Z = \mathbb{E}_p[z]$ and $\frac{d}{d\beta}\mathbb{E}_p[z] = \mathrm{Var}_p(z)$. Differentiate:

$$\frac{dH}{d\beta} = \mathbb{E}_p[z] - \mathbb{E}_p[z] - \beta\, \mathrm{Var}_p(z) = -\beta\ \mathrm{Var}_p(z) \le 0.$$

So entropy falls as β rises, i.e. **rises with** T. The lab checks this numerically (`python prompting_lab.py decode`).

Top-p (nucleus) sampling

Sort tokens by probability, keep the smallest prefix V_p whose mass is at least p, renormalise: $\tilde{p}_i = p_i\, 1[i \in V_p] / \sum_{j \in V_p} p_j$. It removes the long tail of unlikely tokens while adapting the cut-off to how peaked the distribution is.

```
def softmax_T(logits, T):
    z = logits / max(T, 1e-8)
    z = z - z.max()                 # shift-invariance: avoids overflow
    e = np.exp(z)
    return e / e.sum()

def top_p(probs, p):
    order = np.argsort(-probs)
    k = np.searchsorted(np.cumsum(probs[order]), p) + 1
    keep = np.zeros_like(probs, bool); keep[order[:k]] = True
    out = np.where(keep, probs, 0.0)
    return out / out.sum()
```

> **Kahneman link.** At low temperature the model always emits its first impression, like System 1 with no second thought. The slide's illustrative logits put "$0.10" on top: greedy decoding then answers the bat-and-ball question wrongly every time. Diversity (higher T) is what later lets self-consistency find the right answer among several samples.

3. Zero-shot: why specification helps

An instruction-tuned model maps an instruction to behaviour. A complete zero-shot prompt usually has these parts:

```
Role:        who the model is acting as, and for whom
Task:        the exact operation (classify / extract / rewrite / solve)
Context:     facts the model cannot know (domain, audience, data quirks)
Constraints: what must and must not happen; what to do when unsure
Format:      the exact output shape (one word, JSON schema, table)
Input:       the data, clearly delimited
```

Information never hurts on average (derivation)

Let Y be the output, X the short prompt and C extra context. Conditional entropy satisfies

$$H(Y \mid X) - H(Y \mid X, C) = I(Y; C \mid X) \geq 0,$$

because conditional mutual information is a KL divergence, $I(Y; C \mid X) = \mathbb{E}_{X,C}\,\mathrm{KL}\big(p(Y \mid X, C) \,\|\, p(Y \mid X)\big)$, and KL divergence is non-negative (Gibbs' inequality, from Jensen's inequality applied to $-\log$). So on average, adding context narrows the distribution of outputs. Two caveats: (1) "on average" over contexts; a particular misleading sentence can still push a particular output the wrong way; (2) a narrower distribution is only useful if it narrows around the *right* answer.

---

4. Few-shot as Bayesian inference

Brown et al. (2020) showed that GPT-3 learns tasks from k demonstrations in the prompt with no gradient update (in-context learning, ICL). Xie et al. (2022) explain this as implicit Bayesian inference: pretraining documents are generated from latent "concepts" h; a prompt of demonstrations $S = (s_1, \ldots, s_k)$ is evidence about which concept is active.

$$p(y \mid x, S) = \sum_h p(y \mid x, h)\, p(h \mid S), \qquad p(h \mid S) \propto p(h) \prod_{i=1}^{k} p(s_i \mid h).$$

The number game (Tenenbaum, 1999)

Hypotheses are sets of numbers in 1..100 ("even", "powers of 2", "between 10 and 30"). Examples are drawn uniformly from the true set (strong sampling), so

$$p(s_i \mid h) = \frac{1[s_i \in h]}{|h|} \quad \Rightarrow \quad p(h \mid S) \propto p(h)\, \frac{\prod_i 1[s_i \in h]}{|h|^k}.$$

The size principle (derivation)

Take two hypotheses that both contain all k examples, $h_1 \subset h_2$. Their posterior odds are

$$\frac{p(h_1 \mid S)}{p(h_2 \mid S)} = \frac{p(h_1)}{p(h_2)} \left(\frac{|h_2|}{|h_1|} \right)^k .$$

The log odds grow **linearly** in k with slope $\log(|h_2|/|h_1|)$. For $S = \{16, 8, 2, 64\}$, "powers of 2" has 7 members and "even" has 50: the likelihood ratio is $(50/7)^4 \approx 2{,}600$. A handful of well-chosen examples identifies a specific concept; prior differences are overwhelmed quickly.

Posterior predictive

$$p(y \in C \mid S) = \sum_h \mathbb{1}[y \in h]\, p(h \mid S).$$

This is the shading on the slide's grid.

What the result by Min et al. (2022) means in this picture

Replacing demonstration labels with random labels barely hurt accuracy on many classification tasks. In the Bayesian picture, demonstrations carry evidence about the input distribution, the label set and the format, all of which identify the task concept even when individual labels are wrong. Wrong labels do reduce the evidence, so correct labels still matter, especially for tasks the model has not seen.

```
def posterior(D, H, prior):          # H: name → set, prior: name → p(h)
    logp = {n: math.log(prior[n]) - len(D) * math.log(len(ext))
            for n, ext in H.items() if all(d in ext for d in D)}
    m = max(logp.values())
    w = {n: math.exp(l - m) for n, l in logp.items()}   # log-sum-exp for stability
    Z = sum(w.values())
    return {n: v / Z for n, v in w.items()}
```

Practical rules that follow

- **Diversity beats quantity.** Ten near-identical examples are weak evidence against alternative concepts.
- **Cover the label space.** Show each label at least once.
- **Match the format exactly.** The format is part of the concept.

5. Anchoring and contextual calibration

> **Anchoring (Tversky & Kahneman, 1974).** Participants saw a wheel of fortune, rigged to stop at 10 or 65, then estimated the percentage of African countries in the UN. Median estimates were 25 (after 10) and 45 (after 65). An irrelevant number moved the answer.

The live experiment: permutation test

Group A (anchor 10) gives estimates $a_1..a_m$; group B (anchor 65) gives $b_1..b_n$. Test statistic $d = \bar{b} - \bar{a}$. Under the null hypothesis that the anchor has no effect, the group labels are exchangeable. Shuffle the labels R times, recompute $d^{(r)}$ each time, and estimate

$$\hat{p} = \frac{1 + \#\{r : |d^{(r)}| \geq |d|\}}{R + 1}.$$

The $+1$ terms make the estimate valid (never exactly zero). No normality assumption is needed, which suits a small class.

Anchors inside prompts

Zhao et al. (2021) found three biases in few-shot classification: **majority label** (favour labels frequent in the demonstrations), **recency** (favour the last example's label) and **common token** (favour frequent words). Jones & Steinhardt (2022) adapted anchoring and framing tests from psychology and found analogous errors in code-generating models.

Contextual calibration (derivation)

Model the prompt's effect as a multiplicative bias on the label distribution: $\hat{p}(y \mid x) \propto b_y \cdot \ell(y \mid x)$, where ℓ is the evidence from the input and b is the bias the prompt (examples, order, format) adds. For a content-free input such as "N/A", the evidence is uninformative, $\ell(y \mid \mathrm{N/A}) \approx$ constant, so

$$\hat{p}_{\mathrm{cf}} := \hat{p}(y \mid \mathrm{N/A}) \propto b_y.$$

Dividing out the bias recovers the evidence:

$$\hat{q}(y \mid x) = \frac{\hat{p}(y \mid x) / \hat{p}_{\mathrm{cf}}(y)}{\sum_{y'} \hat{p}(y' \mid x) / \hat{p}_{\mathrm{cf}}(y')}, \quad \text{i.e.} \quad \hat{q} = \mathrm{normalise}\left(W \hat{p}\right),\ W = \mathrm{diag}(\hat{p}_{\mathrm{cf}})^{-1}.$$

Zhao et al. present this as an affine transformation fitted with the content-free input, followed by softmax; the diagonal version above is the core idea.

Worked example. $\hat{p}_{\text{cf}} = (0.70, 0.30)$, $\hat{p}(y \mid x) = (0.60, 0.40)$. Then $W\hat{p} = (0.857, 1.333)$, normalised $(0.39, 0.61)$: the decision flips from positive to negative. In the lab's simulation, calibration raises accuracy from 0.71 to 0.79 and restores a balanced prediction rate.

```
def calibrate(p_input, p_content_free):
    q = np.diag(1.0 / p_content_free) @ p_input
    return q / q.sum()
```

```
Template to measure the content-free bias:
<your exact few-shot prompt>
Review: "N/A" →
(read the label probabilities; repeat with "" and "[MASK]" and average)
```

6. Framing and averaging correlated errors

> **Framing (Tversky & Kahneman, 1981).** Facing a disease expected to kill 600 people, 72% chose a sure option when outcomes were described as lives saved; with the same outcomes described as deaths, 78% chose the gamble.

Models are sensitive to frames that carry no meaning at all. Sclar et al. (2024) found that changing separators, casing and spacing in few-shot prompts changed LLaMA-2-13B accuracy by up to 76 points on a task. The remedy is to treat the prompt as a random choice from a set of equivalent framings and average.

Variance of an average of correlated errors (derivation)

Let $\varepsilon_1 .. \varepsilon_K$ be errors with $\operatorname{Var}(\varepsilon_k) = \sigma^2$ and $\operatorname{Cov}(\varepsilon_j, \varepsilon_k) = \rho\sigma^2$ for $j \neq k$. Then

$$\operatorname{Var}\left(\frac{1}{K}\sum_{k}\varepsilon_k\right) = \frac{1}{K^2}\Big[\sum_{k}\operatorname{Var}(\varepsilon_k) + \sum_{j\neq k}\operatorname{Cov}(\varepsilon_j,\varepsilon_k)\Big] = \frac{1}{K^2}\left[K\sigma^2 + K(K-1)\rho\sigma^2\right]$$

$$= \sigma^2\left(\rho + \frac{1-\rho}{K}\right) \xrightarrow{K\to\infty} \rho\sigma^2.$$

Interpretation. Averaging removes the independent part of the error at rate $1/K$ but never the shared part $\rho\sigma^2$. Ten paraphrases of the same flawed prompt share its flaw. This one formula explains prompt ensembling, self-consistency's plateau (§8) and Kahneman's advice to collect independent judgments before discussion.

Prompt ensembling

$$\bar{p}(y \mid q) = \frac{1}{K} \sum_{k=1}^{K} p_\theta(y \mid x_k \oplus q).$$

```
def ensemble(llm_probs, framings, q):
    """llm_probs(prompt) → np.array of label probabilities"""
    P = np.stack([llm_probs(f.format(q=q)) for f in framings])
    return P.mean(axis=0), P.std(axis=0)     # report the spread too
```

Roles and priming: a note on evidence

Kahneman's book gave a chapter to behavioural priming. Many of those studies later failed to replicate, and Kahneman publicly acknowledged relying too much on small studies. For models, a systematic study of 162 roles in system prompts found personas did not reliably improve accuracy on factual questions (Zheng et al., 2024). Use roles to carry audience, standards and constraints, not as a magic word.

7. Chain-of-thought and serial computation

Insert a latent rationale r between question and answer:

$$p(a \mid q) = \sum_{r} p_\theta(r \mid q)\, p_\theta(a \mid q, r).$$

Standard CoT samples or greedily decodes one r and reads off a: a single-sample approximation of the sum. Self-consistency (§8) uses many samples.

Result	Setting	Source
17.9% → 56.9%	GSM8K, PaLM 540B, few-shot CoT vs standard	Wei et al., 2022
17.7% → 78.7%	MultiArith, text-davinci-002, "Let's think step by step"	Kojima et al., 2022

Wei et al. reported that the gains appeared mainly at large model scale; smaller models wrote fluent but wrong chains.

Why writing tokens adds computational power

Multiplying two n-digit numbers involves carries that depend on earlier carries: a chain of serial dependencies whose length grows with n. A fixed-depth network computes each output token with a fixed number L of sequential layer applications. Results of this kind are formalised by Merrill & Sabharwal (2024): transformers that must answer immediately are limited to a weak class of parallel computations, while transformers allowed a number of intermediate decoding steps polynomial in the input length can simulate any polynomial-time computation. Feng et al. (2023) show related results for arithmetic and dynamic programming. The practical reading:

- CoT helps on tasks with serial structure: arithmetic, logic, multi-hop questions, planning.
- CoT does little for pure recall ("capital of Peru"): no serial computation is needed.
- CoT costs output tokens: typically 5–10× the length of a direct answer.

```
Zero-shot CoT
Q: {question}
A: Let's think step by step. End with "Answer: <value>".

Few-shot CoT
Q: {example question 1}
A: {worked reasoning}. Answer: {value}
Q: {example question 2}
A: {worked reasoning}. Answer: {value}
Q: {question}
A:
```

Always ask for a fixed answer marker ("Answer: ..."). It makes extraction reliable, which self-consistency and evaluation depend on.

---

8. Self-consistency: the full analysis

Sample N reasoning paths at temperature around 0.5–0.8, extract answers $a_1..a_N$, return the most common:

$$\hat{a} = \arg\max_a \sum_{i=1}^{N} 1[a_i = a].$$

It estimates the right quantity

Each a_i is a draw from the marginal $p(a \mid q) = \sum_r p(r, a \mid q)$. The vote share $\frac{1}{N}\sum_i 1[a_i = a]$ is an unbiased estimate of $p(a \mid q)$ and converges to it by the law of large numbers. So $\hat{a} \to \arg\max_a p(a \mid q)$: the mode of the *marginal* answer distribution, not the answer of one lucky or unlucky path. Wang et al. (2023) report GSM8K accuracy for PaLM 540B rising from 56.5% (CoT) to 74.4% with 40 sampled paths.

Binary case: exact accuracy

If each path is correct independently with probability p and all wrong paths give the same wrong answer (the worst case), with odd N:

$$\mathrm{Acc}(N) = P\big(\mathrm{Bin}(N, p) > N/2\big) = \sum_{k=\lceil N/2 \rceil}^{N} \binom{N}{k} p^k (1-p)^{N-k}.$$

Hoeffding bound (derivation)

Let $X_i = 1[a_i \text{ correct}] \in \{0, 1\}$, independent, mean $p > \frac{1}{2}$. The vote is wrong only if $\bar{X} \le \frac{1}{2}$, i.e. $\bar{X} - p \le -(p - \frac{1}{2})$. Hoeffding's inequality for bounded independent variables gives $P(\bar{X} - p \le -t) \le e^{-2Nt^2}$. With $t = p - \frac{1}{2}$:

$$P(\text{vote wrong}) \le \exp\big(-2N(p - \tfrac{1}{2})^2\big).$$

Sketch of Hoeffding. For any $\lambda > 0$, Markov's inequality on $e^{-\lambda N(\bar{X} - p)}$ gives $P(\bar{X} - p \le -t) \le e^{-\lambda N t} \prod_i \mathbb{E}\, e^{-\lambda(X_i - p)}$. Hoeffding's lemma bounds each factor by $e^{\lambda^2/8}$ for a variable in $[0, 1]$. Minimising $e^{-\lambda N t + N\lambda^2/8}$ over λ gives $\lambda = 4t$ and the bound e^{-2Nt^2}. This is a quantitative Condorcet jury theorem: if each voter beats chance, majority error decays exponentially. If $p < \frac{1}{2}$, the same argument shows the majority is wrong with probability tending to 1: **voting amplifies a model that is usually wrong**.

Why voting works better in practice: errors scatter

On maths problems, wrong paths rarely agree: one gives 10, another 15, another 4.5. With m equally likely wrong answers, the correct answer only needs to beat each wrong answer individually, roughly $p > (1 - p)/m$. The lab's simulation with $p = 0.45$ and 4 wrong answers: accuracy rises from 0.45 ($N = 1$) to 0.81 ($N = 11$) and 0.99 ($N = 41$), while the "errors agree" case falls to 0.26.

Correlated samples (derivation of the slide formula)

Model correlation as a mixture: with probability ρ, all N samples are copies of one draw (a shared failure mode); with probability $1-\rho$ they are independent. By the law of total probability:

$$\mathrm{Acc}_\rho(N) = \rho \cdot p + (1-\rho)\cdot \mathrm{Acc}_0(N), \qquad \lim_{N\to\infty} \mathrm{Acc}_\rho(N) = \rho p + (1-\rho).$$

The ceiling is below 1 whenever $\rho > 0$ and $p < 1$. To decorrelate: vary the exemplars and their order, paraphrase the instruction, mix models. This is Kahneman's meeting advice in algebra: collect independent judgments before letting them influence each other.

```
from collections import Counter

def self_consistency(llm, prompt, n=10, temperature=0.7):
    answers = []
    for _ in range(n):
        out = llm(prompt, temperature=temperature)
        a = extract_answer(out)            # parse "Answer: ..."
        if a is not None:
            answers.append(a)
    if not answers:
        return None, 0.0
    a, c = Counter(answers).most_common(1)[0]
    return a, c / len(answers)             # vote share is a confidence signal
```

Cost: $N\times$ the tokens of CoT. Gains usually flatten after 10–20 samples. The vote share is a useful confidence score: low agreement flags questions to route to a stronger method or a human.

---

9. Decomposition

Break a problem into sub-questions $s_1 .. s_m$ and solve them in order, each conditioned on earlier answers:

$$p(a \mid q) \approx \prod_{j=1}^{m} p_\theta(s_j \mid q, s_{<j}) \cdot p_\theta(a \mid q, s_{1:m}).$$

If step j succeeds with probability p_j and failures are independent, the chain succeeds with $\prod_j p_j$. Ten steps at 95% give $0.95^{10} \approx 0.60$. Decomposition pays off only when the sub-steps are much easier than the whole problem. On the SCAN length split, least-to-most prompting reached 99.7% versus 16.2% for CoT

with code-davinci-002 (Zhou et al., 2023), because each sub-command is easy and test commands are longer than any example.

```
Least-to-most
{question}
Step 1: List the sub-questions needed to answer this, from easiest to hardest.
Step 2: Answer each sub-question in order. Use earlier answers in later ones.
End with "Answer: <value>".

Plan-and-solve (Wang et al., 2023)
Let's first understand the problem and devise a plan to solve it.
Then let's carry out the plan and solve the problem step by step.

Step-back (Zheng et al., 2024)
What general principle or concept is this question about? State it.
Now use that principle to answer: {question}
```

---

10. Tree of Thoughts and search

A chain commits to its first move. Tree of Thoughts (Yao et al., 2023) treats reasoning as search: at each step the model **proposes** several next thoughts, an **evaluator** (often the model itself, prompted to rate "sure / likely / impossible") scores them, and a search algorithm (BFS with beam b, or DFS with backtracking) expands the best.

Success: one chain vs search

Let ρ_1 be the probability that a single sampled chain solves the problem. With N independent chains and a perfect checker,

$$P(\text{success}) = 1 - (1 - \rho_1)^N,$$

and exhaustive search with a perfect checker succeeds whenever any path works. For the Game of 24 with $\{4, 9, 10, 13\}$, the lab computes $\rho_1 \approx 0.23\%$ for uniformly random moves: a random chain almost never succeeds, while DFS finds 10 − 4 = 6, 13 − 9 = 4, 6 × 4 = 24 in about a thousand node visits. Yao et al. report GPT-4 solving 4% of Game-of-24 problems with CoT and 74% with ToT (breadth 5).

The random-chain probability (exact recursion)

$$\rho(s) = \begin{cases} 1[s = \{24\}] & |s| = 1 \\ \frac{1}{|\mathcal{C}(s)|} \sum_{s' \in \mathcal{C}(s)} \rho(s') & \text{otherwise} \end{cases}$$

where $\mathcal{C}(s)$ are the states reachable by one operation. The lab uses exact `Fraction` arithmetic, which matters for 3 3 8 8 (= 8 ÷ (3 − 8/3)).

Cost

BFS with depth d, beam b and k proposals per state uses about $d \cdot b \cdot k$ proposal calls plus as many evaluation calls: tens to hundreds of calls per problem. Use ToT only for search-shaped problems (puzzles, planning, constrained generation).

```
Propose step
Input: 4 9 10 13
Possible next steps (one operation each, show the numbers left):
10 - 4 = 6 (left: 6 9 13)
...
Value step
Evaluate if the numbers can reach 24 (sure / likely / impossible):
6 9 13 →
```

---

11. Program-aided prompting (PAL)

The model writes a program c; an interpreter executes it: $a = \text{exec}(c),\ c \sim p_\theta(c \mid q)$. Translation is the model's strength; exact arithmetic is not. Gao et al. (2023) report GSM8K accuracy with Codex of 72.0% for PAL versus 65.6% for CoT, and a much larger gap on GSM-Hard, a version with large numbers.

````
{question}
Write a short Python program that computes the answer and stores it in a
variable named answer. Reply with only the code in a ```python block.
````

```
# typical model output for the bat-and-ball item
total, diff = 1.10, 1.00
ball = (total - diff) / 2      # ball + (ball + diff) = total
answer = round(ball, 2)        # 0.05
```

Safety. Model-written code is untrusted. Run it in a sandbox (container, no network, time limit). The lab's `run_pal` uses a tiny restricted namespace for

classroom use only.

12. Self-critique and verifiers

> **WYSIATI: "What you see is all there is."** Kahneman's name for System 1's habit of building the most coherent story from the information at hand and ignoring what is missing.

A model asked to "check your answer" sees exactly the same context as before. Huang et al. (2024) found that intrinsic self-correction, without external feedback, often failed to improve reasoning and sometimes made it worse. Critique helps when the loop adds **new evidence**: test results, interpreter output, retrieved documents (Reflexion, Shinn et al., 2023).

Perfect verifier (derivation)

Draw N independent candidates, each correct with probability p; a perfect verifier accepts exactly the correct ones. We fail only if all N are wrong:

$$P(\text{success}) = 1 - (1-p)^N.$$

With $p = 0.3$, $N = 10$: $1 - 0.7^{10} \approx 0.97$. Plurality voting with three wrong answers gets about 0.38 in the lab's simulation. **Checking beats voting whenever a check exists.**

Imperfect verifier (derivation, and a link to base rates)

Let the verifier accept a correct candidate with probability α (true-positive rate) and a wrong one with probability β (false-positive rate). Process candidates in order and return the first accepted. Each candidate is accepted with probability $\pi = p\alpha + (1-p)\beta$, so

$$P(\text{some candidate accepted}) = 1 - (1-\pi)^N,$$

$$P(\text{correct} \mid \text{accepted}) = \frac{p\alpha}{p\alpha + (1-p)\beta}.$$

The second expression is Bayes' rule with base rate p: exactly the taxi-cab calculation (§14). If the model is rarely right (p small), even a decent verifier accepts mostly wrong answers. Overall success is the product of the two terms.

```
def generate_verify(llm, prompt, check, n=10, temperature=0.8):
    for _ in range(n):
        cand = llm(prompt, temperature=temperature)
        ok, feedback = check(cand)          # e.g. run unit tests, substitute back
        if ok:
            return cand
        prompt += f"\n\nPrevious attempt failed: {feedback}\nTry again."
    return None
```

---

13. Reasoning models

Models such as OpenAI o1 (2024) and DeepSeek-R1 (2025) are trained with reinforcement learning to generate long reasoning before answering, rewarded on verifiable outcomes. A generic objective:

$$\max_{\theta} \; \mathbb{E}_{q\sim\mathcal{D},\,(r,a)\sim\pi_\theta(\cdot|q)}\big[R(q,a)\big] - \beta\,\mathrm{KL}\big(\pi_\theta(\cdot\mid q)\,\|\,\pi_{\mathrm{ref}}(\cdot\mid q)\big).$$

The KL term keeps the policy close to a reference model so it does not drift into degenerate text. DeepSeek-R1 uses GRPO (group relative policy optimisation): sample a group of G outputs per question, and use the group-normalised reward as each output's advantage, $A_i = (R_i - \mathrm{mean}(R))/\,\mathrm{std}(R)$, avoiding a separate value network.

How prompting changes

- "Think step by step" is largely redundant; the model already allocates reasoning.
- The DeepSeek-R1 report observed that few-shot prompting consistently degraded its performance and recommends zero-shot prompts that describe the problem and specify the output format.
- Put effort into the specification: goal, constraints, success criteria, output format.
- Use the provider's effort or thinking-budget setting to trade cost for accuracy; reasoning tokens are billed.

14. Bias as bugs: base rates, conjunction, availability

Base-rate neglect: the taxi cab

85% of cabs are Green, 15% Blue. A witness who is right 80% of the time says "Blue". By Bayes' rule:

$$P(B \mid \text{“B”}) = \frac{P(\text{“B”} \mid B)P(B)}{P(\text{“B”} \mid B)P(B) + P(\text{“B”} \mid G)P(G)} = \frac{0.8 \cdot 0.15}{0.8 \cdot 0.15 + 0.2 \cdot 0.85} = \frac{0.12}{0.29} \approx 41\%$$

Most people answer near 80%, ignoring the base rate. Natural frequencies make it easy (Gigerenzer & Hoffrage, 1995): of 1,000 cabs, 150 are Blue; the witness says Blue for 120 of them and for 170 of the 850 Green cabs; 120 of 290 is 41%.

```
Base-rate-first prompt
Before answering, write:
1. The base rate (prior) for each possibility, with its source in the text.
2. How likely the evidence is under each possibility.
3. Natural frequencies for 1,000 cases.
Then compute the answer from those counts. End with "Answer: <value>".
```

Conjunction fallacy: Linda

For any events A, B: $P(A \cap B) = P(A)P(B \mid A) \leq P(A)$, since $P(B \mid A) \leq 1$. "Bank teller and feminist" cannot be more probable than "bank teller". In Tversky & Kahneman's (1983) studies, a large majority ranked the conjunction higher, because the description is *representative* of a feminist. Binz & Schulz (2023) found GPT-3 made human-like errors on several such vignettes, noting that the famous versions may be memorised. Hagendorff et al. (2023) found intuitive errors on CRT-style items were common in earlier GPT-3 models and largely disappeared in later chat models.

Testing lesson: classic puzzles are in training data. To test reasoning rather than recall, change names, numbers and surface details (the lab's `make_items` does this for bat-and-ball isomorphs).

```
Name the events involved, write the probability rule that relates them,
then decide. Do not rely on how typical the description sounds.
```

Availability and retrieval

People judge frequency by how easily examples come to mind. A model's recall similarly tracks how often a fact appeared in pretraining: Kandpal et al. (2023) found question-answering accuracy falls sharply for facts with few supporting documents. Retrieval-augmented generation replaces recall with reading (Lewis et al., 2020):

$$p(y \mid q) \approx \sum_{d \in \text{top-}k(q)} p_\eta(d \mid q)\, p_\theta(y \mid q, d).$$

```
Answer using ONLY the documents below. Cite the document id for every claim.
If the documents do not contain the answer, reply "not found".

[doc 3] ...
[doc 7] ...

Question: {question}
```

ReAct (Yao et al., 2023) makes this a loop: Thought → Action (search, calculator, code) → Observation → Thought. Each observation is new evidence, the antidote to WYSIATI.

---

15. Automatic prompt optimisation

Treat §1's arg max as black-box optimisation. OPRO (Yang et al., 2024) shows an optimiser model the history of (prompt, score) pairs and asks for a better prompt:

$$x_{t+1} \sim p_{\text{opt}}\big(x \mid \{(x_i, s_i)\}_{i \le t}\big), \qquad s_i = \frac{1}{|D_{\text{dev}}|} \sum_{(q,a) \in D_{\text{dev}}} m\big(f(x_i \oplus q), a\big).$$

```
def opro(score, optimizer_llm, seed_prompts, steps=20, keep=8):
    hist = [(p, score(p)) for p in seed_prompts]
    for _ in range(steps):
        hist = sorted(hist, key=lambda t: t[1])[-keep:]          # best last
        meta = "\n".join(f"text: {p}\nscore: {s:.1f}" for p, s in hist)
        new = optimizer_llm(meta + "\nWrite a new instruction with a higher score.")
        hist.append((new, score(new)))
    return max(hist, key=lambda t: t[1])
```

OPRO found "Take a deep breath and work on this problem step-by-step", reaching 80.2% on GSM8K with PaLM 2-L. Related tools: APE (Zhou et al., 2023) and DSPy (Khattab et al., 2024). **Caution:** this is model selection on a dev set, so it can overfit; keep a held-out test set, and do not expect a found phrase to transfer to other models.

16. Cheat sheet

Situation	Technique	Cost	Mathematical idea
Simple, well-defined task	Zero-shot, full specification	1×	Lower conditional entropy
Unclear format or label space	Few-shot, balanced, shuffled	1×, longer prompt	Bayesian task inference
Reading label probabilities	Contextual calibration	+1 call	Divide out the prompt's prior
Multi-step reasoning	Zero- or few-shot CoT	5–10× output	Latent rationale; serial compute
Answers vary across runs	Self-consistency, decorrelated	N×	Monte Carlo marginal; ρ caps gain
Long compositional problems	Least-to-most, plan-and-solve	1–3×	Product of easy steps
Search or planning puzzles	Tree of Thoughts	10–100×	Search with evaluator
Arithmetic, exact logic	PAL	1× + interpreter	Exact execution
Checkable answers	Generate–verify–revise	N× + checker	$1-(1-p)^N$
Rare or recent facts	RAG, ReAct	+ retrieval	Marginalise over documents
Reasoning model	Zero-shot, clear spec, effort setting	budgeted	RL-trained test-time compute

17. Exercises with solutions

E1. Show that $\mathrm{Var}\left(\frac{1}{K}\sum_k \varepsilon_k\right) = \sigma^2(\rho + \frac{1-\rho}{K})$ for equicorrelated errors, and find the smallest K that gets within 10% of the limit when $\rho = 0.2$.

Solution

Derivation in §6. Need $\frac{1-\rho}{K} \le 0.1\rho$, i.e. $K \ge \frac{1-\rho}{0.1\rho} = \frac{0.8}{0.02} = 40$.

E2. A model's paths are correct with $p = 0.6$. Using Hoeffding, how many independent samples guarantee majority-vote error below 1%?

Solution

$e^{-2N(0.1)^2} \le 0.01 \Rightarrow N \ge \frac{\ln 100}{0.02} \approx 231$. The bound is loose: the exact binomial gives about 1% near $N \approx 130$, and scattered errors need far fewer.

E3. In the number game, how many examples consistent with both "multiples of 10" (10 members) and "even" (50 members) are needed for the odds to favour "multiples of 10" by 1,000:1, with equal priors?

Solution

$5^k \ge 1000 \Rightarrow k \ge \log 1000 / \log 5 \approx 4.3$, so 5 examples.

E4. A verifier has $\alpha = 0.9$, $\beta = 0.2$. Candidates are correct with $p = 0.1$. What fraction of accepted answers are correct? What does this say about using LLM judges on hard tasks?

Solution

$\frac{0.09}{0.09 + 0.18} = \frac{1}{3}$. When the base rate of correct answers is low, even a decent judge accepts mostly wrong answers: the taxi-cab problem again. Improve p (better generation) or β (stricter, external checks).

E5. Calibration: $\hat{p}_{\text{cf}} = (0.5, 0.3, 0.2)$ and $\hat{p}(y \mid x) = (0.45, 0.30, 0.25)$. Which label wins before and after calibration?

Solution

Before: label 1. After: $(0.9, 1.0, 1.25)$ normalised $= (0.29, 0.32, 0.40)$: label 3.

E6. Prove that the entropy of a temperature softmax is non-decreasing in T.

Solution

§2: $dH/d\beta = -\beta \operatorname{Var}_p(z) \le 0$ and $\beta = 1/T$ is decreasing in T.

E7 (lab). Write five new CRT-style questions with fresh numbers and objects. Compare direct, zero-shot CoT, PAL and self-consistency@5 on any model you can access. Report accuracy *and* tokens per item.

Hint

`python prompting_lab.py crt --backend ollama --model <name> --items 30`. Expect direct answering to fall for the System-1 answer more often on small models; PAL should be near-perfect if the model writes valid code.

---

18. Running the lab

```
pip install numpy                       # the only dependency
python lab/prompting_lab.py all         # all offline demos, ~2 seconds
python lab/prompting_lab.py vote        # one section: decode, numbergame, calibrate,
                                        # variance, vote, verify, g24, taxi, crt

# Real model, local (install Ollama and pull a model first):
python lab/prompting_lab.py crt --backend ollama --model llama3.1:8b --items 30

# Any OpenAI-compatible endpoint:
export OPENAI_BASE_URL=https://.../v1  OPENAI_API_KEY=...
python lab/prompting_lab.py crt --backend openai --model <model-name>
```

The `mock` backend is a toy model that answers like System 1 unless prompted to reason; it exists so the evaluation pipeline runs without a network. Its numbers illustrate the pipeline, not any real model.

19. References

Psychology. Kahneman, D. (2011). *Thinking, Fast and Slow*. Tversky & Kahneman (1974), Judgment under uncertainty, *Science*. Tversky & Kahneman (1981), The framing of decisions, *Science*. Tversky & Kahneman (1983), The conjunction fallacy, *Psychological Review*. Frederick (2005), Cognitive reflection and decision making, *J. Economic Perspectives*. Gigerenzer & Hoffrage (1995), *Psychological Review*. Tenenbaum (1999), Bayesian modeling of human concept learning, *NeurIPS*.

Prompting. Brown et al. (2020), *NeurIPS*. Zhao et al. (2021), Calibrate before use, *ICML*. Xie et al. (2022), ICL as implicit Bayesian inference, *ICLR*. Min et al. (2022), *EMNLP*. Wei et al. (2022), Chain-of-thought, *NeurIPS*. Kojima et al. (2022), *NeurIPS*. Wang et al. (2023), Self-consistency, *ICLR*. Zhou et al. (2023), Least-to-most, *ICLR*. Wang, L. et al. (2023), Plan-and-solve, *ACL*. Zheng, H. S. et al. (2024), Step-back, *ICLR*. Yao et al. (2023), Tree of Thoughts, *NeurIPS*. Gao et al. (2023), PAL, *ICML*. Yao et al. (2023), ReAct, *ICLR*. Shinn et al. (2023), Reflexion, *NeurIPS*. Huang et al. (2024), LLMs cannot self-correct reasoning yet, *ICLR*. Lewis et al. (2020), RAG, *NeurIPS*. Zhou, Y. et al. (2023), APE, *ICLR*. Yang et al. (2024), OPRO, *ICLR*. Khattab et al. (2024), DSPy, *ICLR*. Sclar et al. (2024), Prompt formatting sensitivity, *ICLR*. Zheng, M. et al. (2024), Social roles in system prompts, *Findings of EMNLP*.

Theory. Merrill & Sabharwal (2024), The expressive power of transformers with chain of thought, *ICLR*. Feng et al. (2023), Towards revealing the mystery behind chain of thought, *NeurIPS*. Hoeffding (1963), *JASA*.

Models as subjects. Jones & Steinhardt (2022), *NeurIPS*. Binz & Schulz (2023), *PNAS*. Hagendorff, Fabi & Kosinski (2023), *Nature Computational Science*. Kandpal et al. (2023), *ICML*. DeepSeek-AI (2025), DeepSeek-R1, arXiv:2501.12948.